mangen
Архитектура и внутренние контракты

Агибалов Владислав Сергеевич
НИЯУ МИФИ, ИВТ

2026 год

Аннотация

Архитектура UNIX-утилиты `mangen` на C17 и POSIX API: состав модулей, конечные автоматы для сопоставления имён, порядок обработки файлов, инварианты многопоточности, управление ресурсами и модель ошибок.

Содержание

1 Назначение и техническая среда **3**
1.1 Границы модулей 3

2 Обнаружение и фильтрация **4**
2.1 Обход каталогов 4
2.2 Ссылки и изменяемая файловая система 4
2.3 Matcher: модель wildcard-шаблона 5

3 Очередь и многопоточность **8**
3.1 MPMC 8
3.2 Условия отсутствия гонок 8

4 Каталог и вычисление SHA-2 **9**
4.1 Владение ресурсами 9
4.2 Состояние записи 9
4.3 Потоковое хэширование 9

5 Формат манифеста и ошибки **9**

6 Сложность и ограничения **10**

7 Проверяемая переносимость **10**

Список литературы **11**

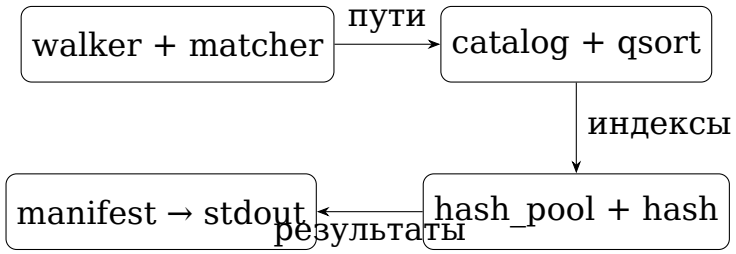
1. Назначение и техническая среда

Утилита формирует детерминированный манифест пар «относительный путь --- криптографическая хэш-сумма содержимого». Корневой каталог задаётся позиционным аргументом или по умолчанию равен текущему. Результат выводится в `stdout`, сообщения об ошибках --- в `stderr`.

Собственная производственная часть использует C17 (ISO/IEC 9899:2018) и POSIX-интерфейсы. Сторонние компоненты: Concurrency Kit 0.7.2, Mbed TLS 3.6.7 LTS, GoogleTest 1.18.0 (только тесты). Сборка должна работать без сетевых загрузок на macOS и Linux с поддерживаемыми архитектурами процессора.

1.1. Границы модулей

Модуль	Основная ответственность
`matcher`	Компиляция wildcard-шаблонов, NFA, DFA, минимизация и сопоставление полного базового имени.
`queue`	Многопоточная MPMC-очередь заданий, жизненный цикл, согласованная обработка заполнения.
`catalog`	Локальные наборы путей, перенос владения, сортировка, состояния обработки и ожидание результата.
`hash`	Потоковое SHA-256, SHA-384, SHA-512 и перевод digest в HEX.
`hash_pool`	Распределение индексов каталога между хэширующими рабочими потоками.
`walker`	Рекурсивное обнаружение файлов и каталогов с применением фильтров.
`manifest`	Ожидание упорядоченных результатов, экранирование имён и запись в stdout.
`app`	Обработка CLI, объединение библиотек, итоговый статус процесса и сигналов ОС.



Две рабочие стадии выполняются последовательно: сначала все пути обнаруживаются и глобально сортируются, затем независимо вычисляются хэши. Печатать сортированные готовые результаты допускается до завершения всех вычислений.

2. Обнаружение и фильтрация

2.1. Обход каталогов

Каждая рабочая единица содержит каталог для обследования. Walker-поток берёт задание, открывает свой дескриптор каталога, перебирает элементы, фильтрует базовые имена, добавляет подкаталоги в очередь, а имена файлов в свой локальный массив. После завершения потоков локальные массивы объединяются и сортируются.

Счётчик незавершённых задач увеличивается перед публикацией каждого нового каталога и уменьшается после окончания его обработки. Нулевой счётчик одновременно означает, что нет ожидающих и активных заданий. Само наличие временно пустой очереди недостаточно для завершения потоков.

2.2. Ссылки и изменяемая файловая система

Ссылка на обычный файл включается под именем ссылки, её содержимое получается по целевому файлу. Ссылка на каталог не обходится; для неё выдаётся диагностика. Повреждённые ссылки и ошибки чтения фиксируются; доступная часть дерева продолжает обрабатываться.

Проверка типа объекта и последующее открытие могут наблюдать разные состояния при конкурентном изменении файловой системы. Применяются `lstat`, `fstatat`, `fstat` и контроль результата открытия, но `mangen` не создаёт атомарный снимок файловой системы.

2.3. Matcher: модель wildcard-шаблона

Модуль `matcher` проверяет имя файла целиком.

Шаблон заранее преобразуется в конечный автомат.

Единица сопоставления --- расширенный графемный кластер Unicode, то есть воспринимаемый пользователем символ, который может занимать несколько кодовых точек и байтов UTF-8. Например, буква вместе с комбинируемым акцентом считается одним кластером. Разбиение выполняет `utf8proc`; корректные участки UTF-8 нормализуются в форму NFC. Некорректные байты UTF-8 сохраняются и при разбиении образуют отдельные единицы. Сопоставление чувствительно к регистру.

Элемент шаблона	Действие
Литерал	Совпадает с соответствующим графемным кластером после NFC.
.	Совпадает ровно с одним графемным кластером.
*	Совпадает с произвольным числом кластеров, включая ноль.
\	Экранирует следующий графемный кластер, превращая его в литерал.

Несколько шаблонов `-e` применяются независимо: имя исключается, если хотя бы один скомпилированный шаблон совпал с ним. Сопоставляется только `basename`, а не полный путь. Одинаковые соседние звёздочки сворачиваются в одну при разборе; завершающая обратная косая черта считается некорректным шаблоном.

2.3.1. Математическая семантика

Обозначим через Σ алфавит нормализованных графемных кластеров, дополненный отдельными некорректными байтами UTF-8. Через ε обозначим пустую последовательность, а через $L(p)$ --- язык, описываемый шаблоном p. Для литерала $g \in \Sigma$ выполняется:

$$L(\varepsilon) = \{\varepsilon\}, \qquad L(g) = \{g\},$$
$$L(\,.\,) = \Sigma, \qquad L(*) = \Sigma^*,$$
$$L(pq) = L(p) \cdot L(q).$$

Здесь $\cdot$ означает конкатенацию языков; для экранированного символа применяется правило литерала. Если после NFC входное имя разбито на $x = g_1 g_2 \dots g_m$, то результат равен

$$\text{match}(p, x) \iff x \in L(p).$$

Поэтому шаблон `a*b` принимает `ab`, `axb` и `axxxb`, но отвергает `aba`; шаблон `a.b` принимает между `a` и `b` ровно один кластер. Это полное совпадение, а не поиск подстроки.

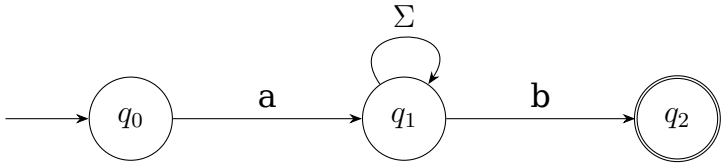
2.3.2. Разбор и построение NFA

Компиляция начинается с токенизации `MATCHER_TOKEN_LITERAL`, `ANY_ONE`, `ANY_MANY`. Непрерывные литералы нормализуются в NFC и разбиваются на кластеры. Далее `matcher_nfa_build` создаёт недетерминированный конечный автомат $N = (Q, \Sigma, \delta, q_0, F)$ без явных ε-переходов.

Пусть r --- число токенов, потребляющих ровно один кластер (литерал или точка). Тогда $Q = \{q_0, \dots, q_r\}$, $F = \{q_r\}$. После каждого потребляющего токена текущий номер состояния увеличивается:

$\delta(q_i, g) \supseteq \{q_{i+1}\}$	для литерала g,
$\delta(q_i, c) \supseteq \{q_{i+1}\}$	для точки, любого $c \in \Sigma$,
$\delta(q_i, c) \supseteq \{q_i\}$	для звёздочки, любого $c \in \Sigma$.

Звёздочка не добавляет состояние. Её петля позволяет поглощать произвольное число кластеров, а переход следующего токена доступен из того же состояния, поэтому нулевое число кластеров также допустимо.



NFA для шаблона `a*b`.

2.3.3. Детерминизация и алфавитные классы

Алгоритм подмножеств преобразует NFA в DFA. Состояние DFA представляет множество активных состояний NFA, изначально $S_0 = \{q_0\}$; после прочтения кластера c вычисляется

$$\Delta(S, c) = \bigcup_{q \in S} \delta(q, c), \qquad \text{accept}(S) \iff q_r \in S.$$

Число возможных подмножеств в общем случае достигает 2^{r+1}, но компилятор прекращает построение при превышении лимита состояний. Множества реализованы битовыми векторами `uint64_t`; переходы по точке и петлям звёздочки рассчитываются битовыми масками и сдвигами, а одинаковые множества переиспользуются через хэш-таблицу.

Поскольку конкретный литерал важен для перехода, а все остальные кластеры одинаковы по отношению к литералам шаблона, используется сжатый алфавит. Пусть $g_1, \ldots, g_k$ --- различные литеральные кластеры шаблона. Функция класса

$$\kappa(c) = \begin{cases} i, & c = g_i, \quad 1 \le i \le k, \\ 0, & c \notin \{g_1, \ldots, g_k\} \end{cases}$$

сводит потенциально неограниченный алфавит к $k+1$ классам. Литералы хранятся в отсортированном массиве; для выбора класса используется двоичный поиск. Переходы DFA хранятся в плотной таблице; индекс элемента вычисляется по формуле

$$i = s \cdot k_{\text{classes}} + c,$$

где s --- текущее состояние, c --- класс входного кластера.

2.3.4. Минимизация DFA

`matcher_min_dfa_build` удаляет различия между состояниями, которые принимают одинаковые множества продолжений. Сначала состояния разбиваются на принимающие и непринимающие. Далее алгоритм уточнения разбиения разделяет блоки по обратным переходам для каждого класса алфавита, пока разделение более невозможно. Для быстрого поиска предшественников построена структура CSR --- массив смещений и смежный массив исходных состояний. Равные по поведению состояния заменяются одним состоянием минимального DFA; его начальное состояние получает индекс ноль.

2.3.5. Сопоставление и оценка ресурсов

Функция `mangen_matcher_compile` выполняет четыре этапа: разбор шаблона, построение NFA, детерминизацию и минимизацию DFA. Временные представления освобождаются, а объект `mangen_matcher_t` владеет только минимальным DFA. Для нового имени `mangen_matcher_match` нормализует вход, выделяет кластеры, определяет класс κ и делает один переход DFA

на кластер. Для ASCII-строк предусмотрен путь без выделения буфера NFC.

При m кластерах входа и k различных литералах стоимость прохода после нормализации --- $O(m \log(k+1))$ сравнений классов и $O(m)$ табличных переходов; таблица переходов занимает $O(s(k+1))$ элементов, где s --- число состояний минимального DFA. Компиляция затратнее сопоставления, поэтому один объект выгодно переиспользовать для множества имён. Реализация ограничивает входной шаблон 1024 байтами и промежуточный DFA 4096 состояниями; превышение лимита возвращает `MANGEN_MATCHER_LIMIT_EXCEEDED`.

API различает отсутствие совпадения и внутреннюю ошибку через `mangen_matcher_match`; сокращённый вызов `mangen_matcher_matches` возвращает `false` в обоих случаях. Скомпилированный объект при проверке не изменяется; его освобождение через `mangen_matcher_free` нельзя выполнять одновременно с активными проверками по тому же указателю.

3. Очередь и многопоточность

3.1. MPMC

`ck_ring` используется для обмена заданиями между несколькими производителями и потребителями. Состояния «закрыто» и «отменено» отличаются: закрытие прекращает приём новых задач, но допускает выгрузку ожидающих; отмена требует остановки и соединения всех потоков перед освобождением памяти.

Функция создания очереди задаёт минимальную ёмкость; она может округляться согласно ограничениям `ck_ring` и остаётся постоянной на протяжении жизни объекта. Операция `try_push` не выделяет память и при заполнении возвращает статус `MANGEN_QUEUE_FULL`; элемент остаётся у вызывающего кода. Успешная запись передаёт владение очереди, успешное извлечение --- потребителю.

Освобождение очереди допускается только после завершения всех потоков, которые к ней обращаются.

3.2. Условия отсутствия гонок

1. Каждый каталог обрабатывается ровно одним владельцем задания.

2. Каждый индекс хэширования выдаётся не более одному worker-потоку.
3. Хэш и ошибка публикуются до перехода записи в терминальное состояние.
4. Поток печати ждёт результата по условной переменной; активное ожидание исключено.
5. Освобождение каталога и очереди разрешено только после завершения пользующихся ими потоков.

4. Каталог и вычисление SHA-2

4.1. Владение ресурсами

После добавления пути локальный batch владеет строкой. При успешном объединении владение получает каталог; он отвечает за освобождение строк. Указатель на запись из каталога является заимствованным и действителен до удаления каталога. Структуры закрыты непрозрачными C-типами.

4.2. Состояние записи



`SUCCESS` содержит валидные бинарные байты digest длины 32, 48 или 64, `ERROR` содержит причину отказа. Ни один поток не должен печатать незавершённый результат или использовать фиктивный хэш при ошибке.

4.3. Потоковое хэширование

Mbed TLS получает содержимое файла блоками фиксированного размера. Операции `open/read/close` проверяются, размер файла не определяет размер оперативной памяти хэширования. SHA-256 возвращает 64 HEX-символа, SHA-384 --- 96, SHA-512 --- 128; буквенные символы A--F заглавные. Стандарт хэш-функций --- NIST FIPS 180-4.

5. Формат манифеста и ошибки

```
relative/path.txt : A3F1...<FULL_HEX_DIGEST>
```

Разделитель --- ровно `␣:␣` с пробелом с обеих сторон. В пути экранируются обратная косая черта, LF и CR. Сортировка должна быть глобальной, лексикографической по байтам относительного пути. Сообщения диагностики идут в `stderr` независимо от `stdout`. Статусы процесса: `0` --- успешный проход, `1` --- ошибка выполнения или частичный результат, `2` --- ошибка CLI.

6. Сложность и ограничения

Пусть F --- количество обнаруженных файлов, D --- количество каталогов, S --- совокупный размер прочитанного содержимого. Обнаружение требует $O(F + D)$ посещений, сортировка --- $O(F \log F)$ сравнений путей, хэширование --- $O(S)$ работы по обработке байтов. В памяти остаётся $O(F)$ записей и путей, рабочая очередь и ограниченные буферы потоков. Пиковое потребление зависит от длин путей и числа очередных задач.

7. Проверяемая переносимость

На macOS и Linux нужны POSIX threads, атомарные операции C17 и корректная сборка Concurrency Kit для данной архитектуры. В CMake следует использовать `find_package(Threads REQUIRED)` и цель `Threads::Threads`. Для `realpath(path, NULL)` на glibc при строгом C17 нужны подходящие feature-test macros (например, `_POSIX_C_SOURCE=200809L`), если данный вызов действительно используется.

Подтверждённые пользовательским журналом результаты macOS и проверки Linux разграничены в документе `verification.pdf`.

Список литературы

[1] ISO/IEC 9899:2018. Programming languages --- C.

[2] The Open Group. POSIX Base Specifications. https://pubs.opengroup.org/onlinepubs/9799919799/

[3] NIST. FIPS 180-4. Secure Hash Standard. https://csrc.nist.gov/pubs/fips/180-4/upd1/final

[4] Concurrency Kit. https://github.com/concurrencykit/ck

[5] Mbed TLS. https://github.com/Mbed-TLS/mbedtls

[6] CMake. FindThreads. https://cmake.org/cmake/help/latest/module/FindThreads.html